澳門大學

UNIVERSIDADE DE MACAU

UNIVERSITY OF MACAU

iSuperviz: Your AI Research Supervisor

CISC3003Team Project Report

Team 07 — iSuperviz

Submission Deadline: 2026 May 10, 11:00 pm

Deployed URL: https://elsayx.github.io/CISC3003-dc328669-2026GitHub/CISC3003-TeamProject-Team07-iSuperviz/

GitHub Archive URL: https://github.com/Elsayx/CISC3003-dc328669-2026GitHub/tree/master/CISC3003-TeamProject-Team07-iSuperviz

Pair	Student ID	Name	Team Role
Pair 08	DC328669	Yang Xu	Team Lead / Member
Pair 08	DC328023	Jiang Xingyu	Partner
Pair 12	DC326312	Huang Sofia	Member
Pair 12	DC326351	Fan Zou Chen	Partner
Pair 04	DC227126	Si Tin Iek	Member
Pair 04	DC226328	Ma Iat Tim	Partner

Project Abstract

iSuperviz is Team 07's full-stack responsive web application for CISC3003, designed to help university researchers manage the continuous flow of academic papers from discovery to reflection, idea development, and credit-based service usage. The project starts from the course requirement that every team system should demonstrate testable responsive design, user registration with email verification, login and forgotten-password recovery, a dashboard with search, history and shopping-cart services, team-defined peculiar services, and a deployed Web URL for public testing. Our response is an AI research supervisor platform built with React, TypeScript, Ant Design, React Flow, Node.js, Express, SQLite and a GitHub Pages-compatible mock API layer. Users can sign up, log in with the demo account, set research preferences, browse a seeded arXiv-style paper catalogue, explore reflective notes, ask an AI assistant panel, download PDFs with credit deduction, save ideas, inspect idea-paper relationships in a graph, run a hallucination audit, top up credits in a research store, check out a cart, and review orders and activity history in the dashboard. The frontend was designed for desktop and mobile breakpoints, while the backend defines structured data tables for users, captchas, products, carts, orders, search history, papers, notes, ideas, related papers, chat messages, feedback and audit jobs. Because GitHub Pages cannot run a Node server, the deployed bundle includes a client-side mock adapter that mirrors the Express API, allowing assessors to test the complete workflow through one public URL. The project deliverables include source code, static deployment, proposal, presentation, screenshots, tests, report and digital-story materials. This report documents the project context, goals, services, tools, task division, evaluation, learning outcomes, public-forum responses, installation steps, remaining limitations, citations and AI-usage declaration. Overall, iSuperviz demonstrates how a student team can combine practical web engineering, academic workflow design, transparent documentation, and deployment evidence into one assessable artifact before submission.

Table of Contents

Project Abstract 2

1. Project Context - Full-Stack Web Applications with RWD and Deployed URL 4

2. Project Goals 9

3. Project Services and Data Requirements 10

4. Project Tasks and Tools 10

5. Project Challenges 11

6. Project Deliverables 11

7. Project Division of Work among Team Members 12

8. Project Contributions from Individual Team Members 12

9. Project Criteria of Success 13

10. Project Evaluation 14

11. Project Learning - Individual and Team 14

12. Project Test Case(s) for Functional Requirements 15

13. Project Demo Walkthrough (Screen Shots) 15

14. Public Forum Q&A Responses from the Remaining Six Teams 23

15. Work-in-Progress Details throughout the Project Period for the Collaborative Forum 23

16. Installation Instructions, Deployment Instructions and Data Usage 24

17. Project Incomplete - What Was Not Completed by the Submission Deadline 24

18. Project Citations and References (APA Style) 24

19. Project Declaration of AI Usage 25

20. Remarks 25

Final Requirement Coverage Checklist 26

1. Project Context - Full-Stack Web Applications with RWD and Deployed URL

Responsive Web Design (RWD) evidence:

Figure 1A shows that iSuperviz can adapt to multiple screen sizes, including desktop, laptop, tablet, and mobile views. This supports the RWD requirement in the project brief. The same deployed website keeps the main home-page content readable and usable across different devices, proving that the frontend is not designed only for one fixed desktop resolution.

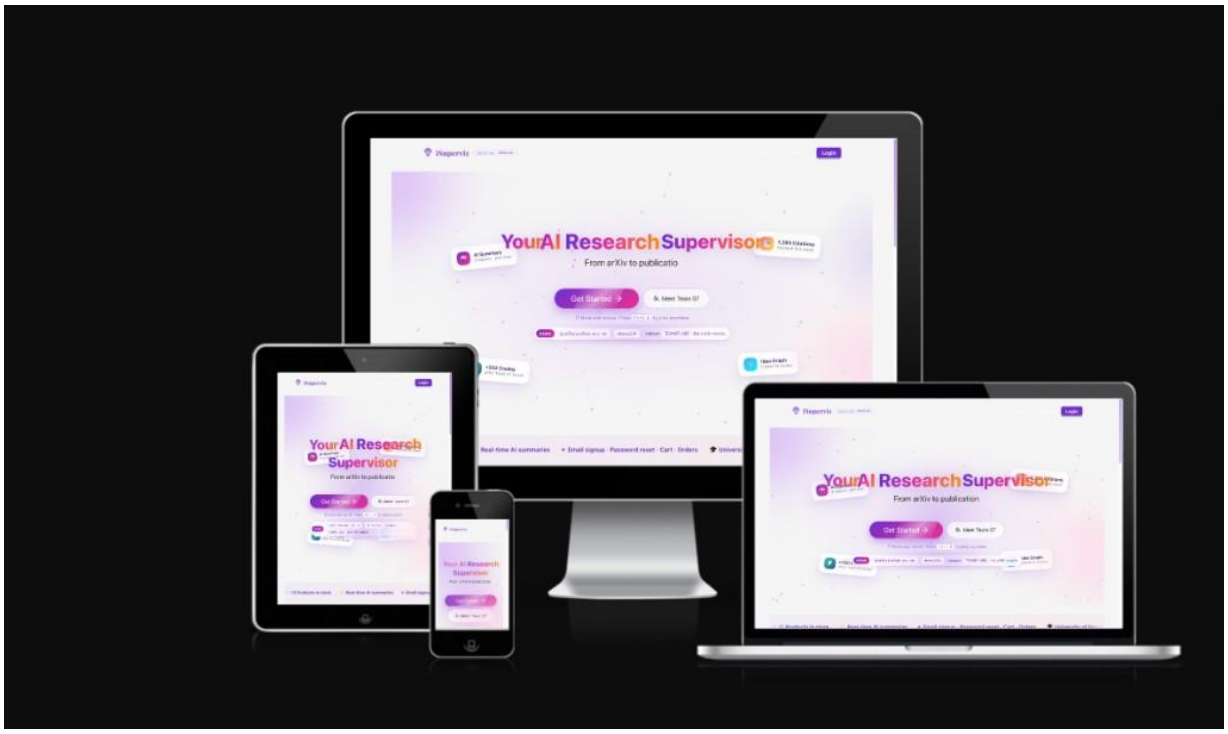

Figure 1A. Responsive Web Design evidence: iSuperviz home page shown across desktop, laptop, tablet, and mobile screen sizes.

The screenshot below is the core checklist used to introduce and evaluate this project. We use it as a direct reference point, then explain how iSuperviz satisfies each item through actual website functions and deployable artifacts.

- Project requirements must incorporate the following:
  - testable responsive design for your project system (front-end)
  - design and implementation of full-stacked web applications for your project system, covering the following essential service modules:
    - user signup/email verification
    - user login/password reset (forgotten)
    - user dashboard service to allow individual users to manage his or her activities, including:
      - the search-related services from your system
      - the history-related services from your system
      - the shopping cart related services from your system
      - your *proposed peculiar services* as expected from your team's project ideas, not excluding any e-commerce related services, if your system domain covers any retail-oriented businesses.
- Project deployment requires the provision of a Web URL to demonstrate your project work - very important means for assessment of your team and individual contributions from the project assignment, as evidence of course learning.

Figure 1. Course requirement screenshot: responsive design, full-stack services, dashboard/search/history/cart/peculiar services and deployed URL.

Requirement-function screenshots placed immediately after Figure 1:

The screenshots below are arranged in the same logic as the course requirement list: account services first, then dashboard search/history/order management, then shopping-cart service. Together they prove that the functions mentioned in Figure 1 are visible and testable in the deployed website.

This screenshot directly answers the dashboard/search/history requirement in Figure 1. The user can search system records, see matching results, and keep timestamped history entries in the personal dashboard.

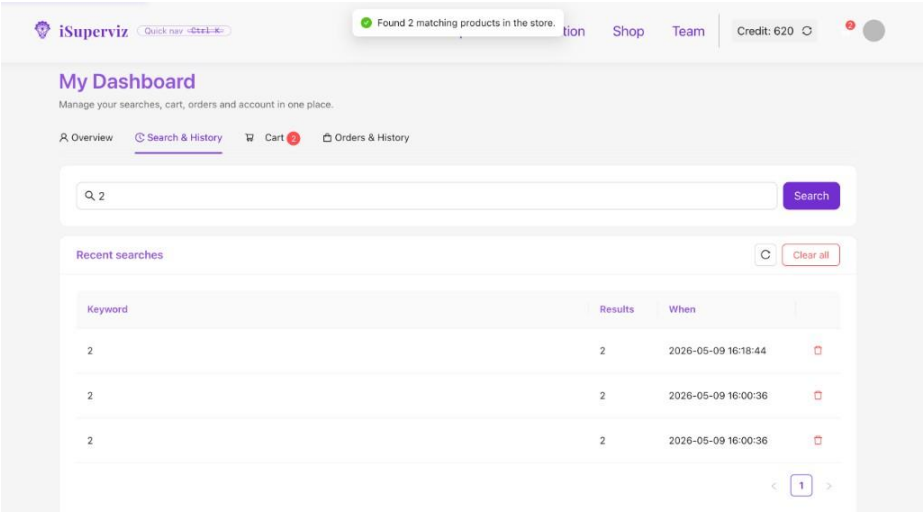

Figure 1.1. User dashboard search-related and history-related service.

This screenshot shows the required user login module. The demo account professor@um.edu.mo / demo1234 allows assessors to enter the authenticated workflow quickly.

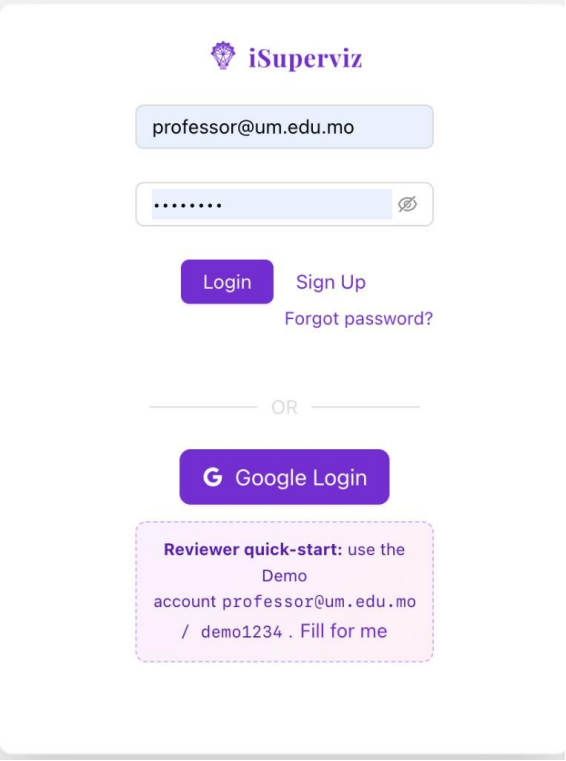

Figure 1.2. User login service.

This screenshot shows the forgotten-password service required in Figure 1. Users can enter their registered email and choose a new password through the reset form.

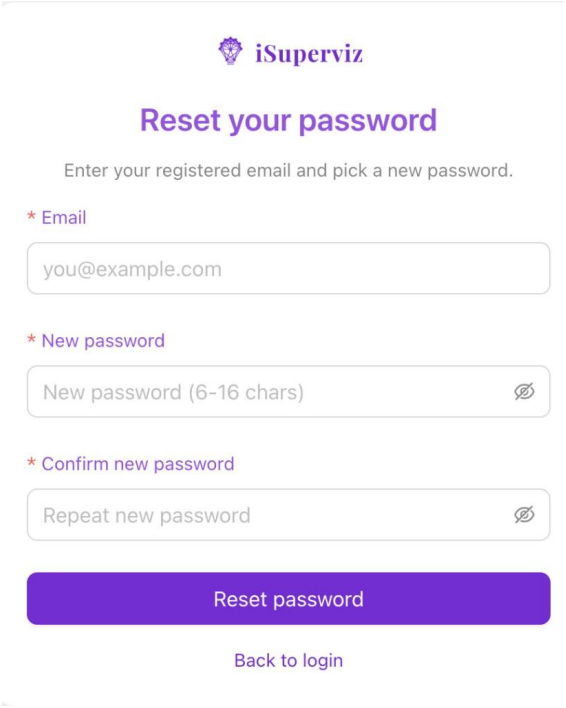

Figure 1.3. Forgotten-password reset service.

This screenshot shows the user sign-up module. For the demo, account creation is simplified with nickname, email and password, while the source backend also includes captcha/email-verification logic with Nodemailer fallback.

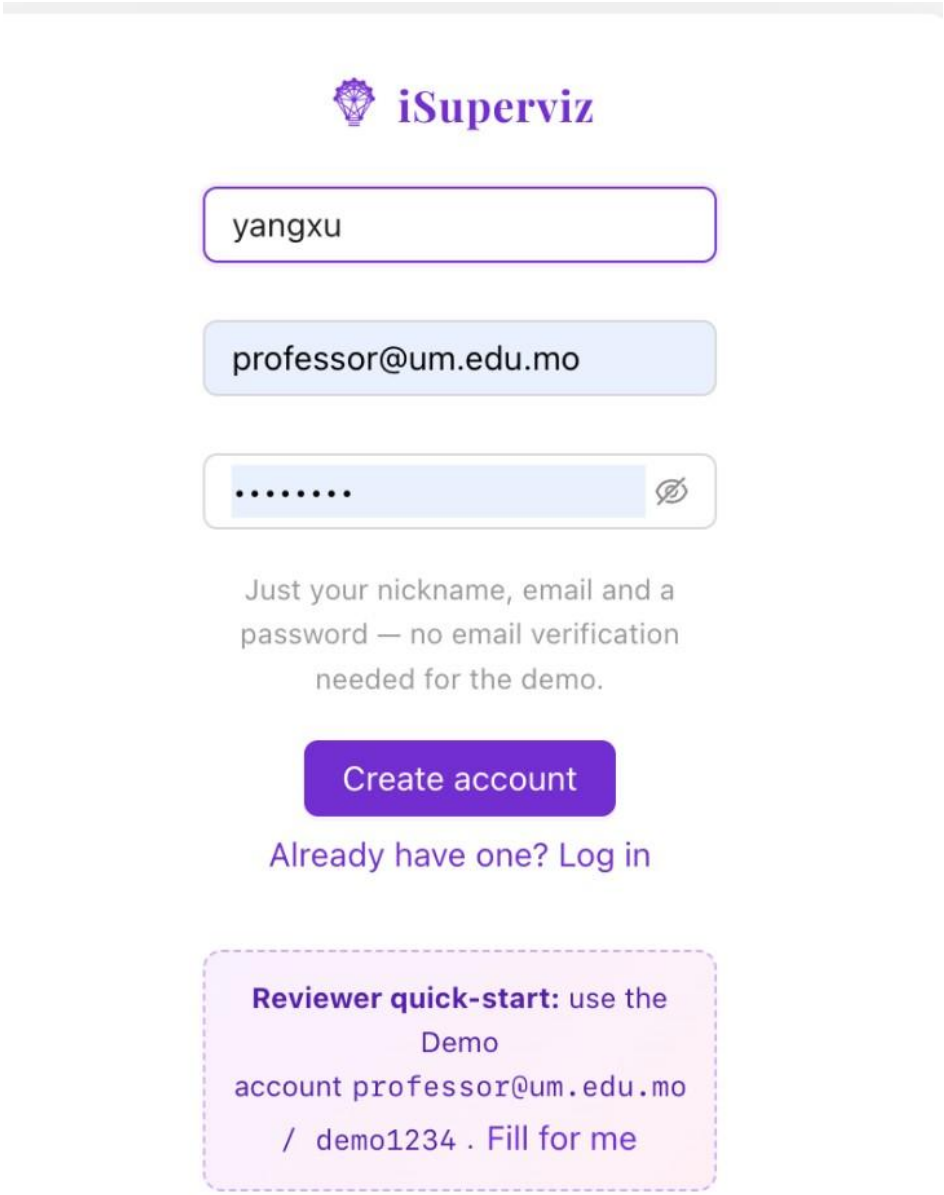

Figure 1.4. User sign-up service.

This screenshot shows the individual dashboard service for managing activities. Orders and History are presented together, covering order records, payment status and activity records.

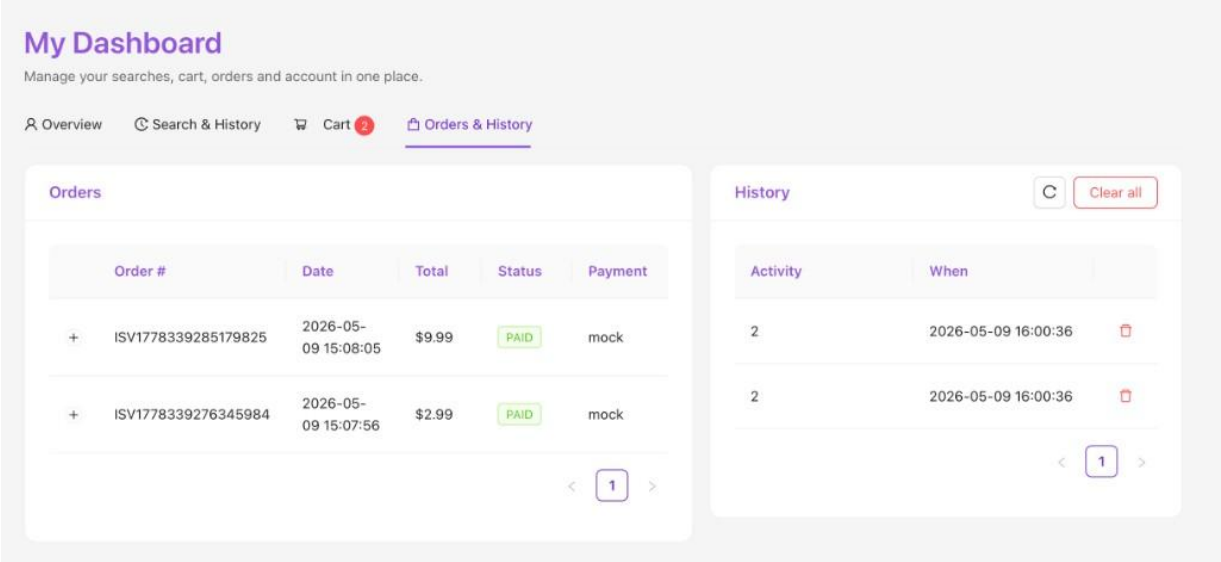

Figure 1.5. Dashboard orders and activity history.

This screenshot shows the shopping-cart related service. Users can search products, filter categories, add credit packs to the cart and proceed toward checkout.

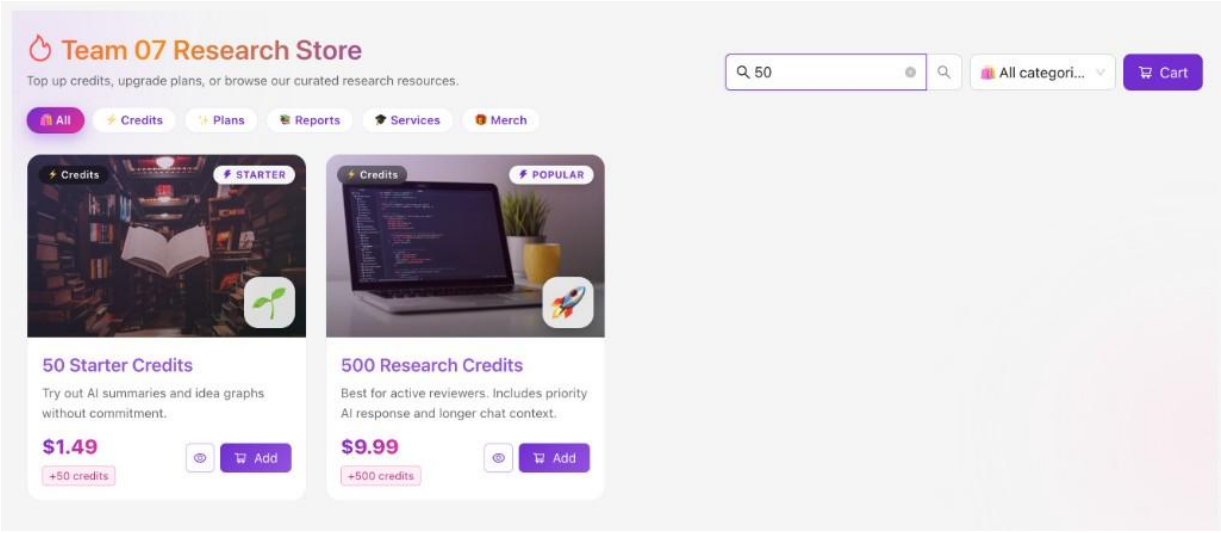

Figure 1.6. Shopping-cart related service through the research store.

Reading Figure 1 against our website:

- "Testable responsive design" is addressed through React/TypeScript pages using Ant Design grid layouts and responsive tests at mobile, tablet and desktop widths.
- "Full-stacked web applications" is addressed by a React frontend plus a Node.js/Express/SQLite backend in the coding archive.
- "User signup/email verification" appears in the authentication service using captcha/email-verification logic with Nodemailer and console fallback.
- "User login/password reset" appears in the login and forgotten-password pages and the related backend endpoints.

- “User dashboard service” appears in the dashboard page, where one user can manage credits, search history, cart summary and orders.
- “Search-related services” are shown in paper search and product search, both recorded as history.
- “History-related services” are shown in search/activity history and order history.
- “Shopping cart related services” are shown in the research store, cart, checkout and orders.
- “Proposed peculiar services” are shown in AI Paper Chat, Idea Graph, Hallucination Audit and credit-based PDF/Explore actions.
- “Project deployment” is shown by the public GitHub Pages URL, allowing the project to be tested through one web link.

Context Summary

iSuperviz is therefore presented as a full-stack research-workflow system, not just a static page. The static GitHub Pages version uses a mock API adapter only because GitHub Pages cannot execute Node.js, while the source archive includes the real Express/SQLite backend.

Requirement from Brief	iSuperviz Implementation Evidence
Responsive frontend	React + TypeScript + Ant Design layouts; Jest responsive tests.
Full-stack design	React SPA frontend; Express/SQLite backend; GitHub Pages mock API parity.
Signup/email verification	/api/sendCaptcha and /api/login with Nodemailer fallback.
Login/password reset	Forgot password, account check, reset and change-password endpoints.
Dashboard/search/history/cart	Dashboard, product/paper search, search_history, cart, checkout and orders.
Peculiar services	AI Paper Chat, Idea Graph, Hallucination Audit, credit-based PDF/Explore.
Deployment URL	https://elsayx.github.io/CISC3003-dc328669-2026GitHub/CISC3003-TeamProject-Team07-iSuperviz/

2. Project Goals

- Help researchers track recent academic papers that match their areas, topics and language preferences.
- Provide paper summaries, reflective notes and suggested ideas so reading papers feels like interacting with a research supervisor.
- Visualise saved ideas and related papers through an Idea Graph that supports knowledge exploration.
- Offer a credit-based e-commerce workflow for top-ups, plans and research-related products.

- Demonstrate all required full-stack service modules through a deployed URL that assessors can test directly.
- Keep the project explainable through screenshots, tests, source code, report, PPT and forum responses.

3. Project Services and Data Requirements

The service design follows the modules in Figure 1 and extends them into a research workflow domain. Data is stored in SQLite for local backend use and mirrored in the mock database for GitHub Pages testing.

Service Area	Main API / Data Requirement	Purpose
Authentication	users, captchas; /api/sendCaptcha, /api/login	Register users and verify email/captcha.
Password Recovery	forgot_password, check_account, reset_password, change_password	Recover forgotten passwords.
Profile and Credits	users.credit; edit_profile, get_credit, consume_credit	Track preferences and credit usage.
Paper Tracking	papers, paper_notes; get_paper_info, get_paper_notes	Search and read papers.
Idea Services	ideas, idea_related_papers	Save ideas and connect papers.
AI Chat	chat_messages	Paper-specific Q&A.
Search History	search_history	Record search and activity history.
Shopping Cart	products, cart_items, orders	Cart, checkout and orders.
Hallucination Audit	hallucination_jobs	Run peculiar review/audit service.

Open or seeded project data includes the paper catalogue, product catalogue, demo account, team roster, screenshots and deterministic mock audit outputs. No sensitive real user data is required for assessment.

4. Project Tasks and Tools

Task Stage	Main Work	Tools / Technologies
Planning and proposal	Define problem, goals, modules and success criteria.	Word proposal, course brief, team discussion.
Frontend implementation	Build Home, Team, Login, Setting, Paper, PaperDetail, IdeaGraph, Hallucination, Shop, Cart, Dashboard.	React, TypeScript, Ant Design, React Router, React Flow.

Backend implementation	Implement auth, reset, history, cart, orders, papers, ideas and audit APIs.	Node.js, Express, SQLite, better-sqlite3, Nodemailer.
Static deployment	Mirror backend endpoints for GitHub Pages.	Axios mock adapter, GitHub Pages.
Testing and QA	API parity, seed data and responsive rendering tests.	Jest, React Testing Library, manual browser testing.
Documentation	Proposal, PPT, report, screenshots, forum responses.	PowerPoint, Word, GitHub.

5. Project Challenges

- GitHub Pages cannot run Node.js; we solved this with a client-side mock API while keeping the real backend in source.
- Email verification needs SMTP configuration; we added Nodemailer with console fallback for reliable demos.
- The Idea Graph needed meaningful data without real users; seeded related papers make the graph visible.
- Credit deduction had to be visible; Explore and Download PDF now show "-10 credits" and update balance/history.
- Dashboard scope could be misunderstood; Orders & History now presents orders and activity history together.
- Dense pages such as PaperDetail and IdeaGraph required responsive layout and manual QA.

6. Project Deliverables

Deliverable	File / URL / Evidence	Status
Project proposal	iSuperviz_Team07_Proposal (1).docx	Prepared
Project PPT	CISC3003_Team07_modified(1).pptx	Prepared
Full source code	iSuperviz-full-source-clean.zip	Prepared
Static deployed artifact	CISC3003-TeamProject-Team07-iSuperviz directory	Published
Live testing URL	https://elsayx.github.io/CISC3003-dc328669-2026GitHub/CISC3003-TeamProject-Team07-iSuperviz/	Published
GitHub archive link	https://github.com/Elsayx/CISC3003-dc328669-2026GitHub/tree/master/CISC3003-TeamProject-Team07-iSuperviz	Published
Screenshots	12 walkthrough screenshots	Embedded
Functional tests	3 suites, 18 tests passed	Verified
Project report	iSuperviz_Team07_Project_Report_Pretty.docx	Generated
Digital stories	Team and individual videos	To be submitted separately

7. Project Division of Work among Team Members

Member	Pair	Assigned Work for Final Submission
Yang Xu	Pair 08	Overall lead. Completed remaining project work not otherwise listed, including integration, deployment, report, GitHub update, archive organisation and major implementation coordination. Worked with Si Tin Iek on frontend and backend.
Jiang Xingyu	Pair 08	Handled public forum communication: asking questions, answering questions from other teams and supporting written forum responses.
Huang Sofia	Pair 12	Prepared project PPT materials, visual presentation structure and slide-level feature explanation.
Fan Zou Chen	Pair 12	Prepared project PPT materials together with Huang Sofia, including presentation flow and deliverable explanation.
Si Tin Iek	Pair 04	Worked with Yang Xu on frontend and backend development and service integration.
Ma Iat Tim	Pair 04	Prepared citation/reference-related materials and APA reference organisation.

8. Project Contributions from Individual Team Members

Yang Xu (DC328669) — Team Lead

Served as team lead and completed most remaining development and submission work. Yang Xu coordinated the project direction, integrated the full-stack application, updated the deployed GitHub Pages build, organised the coding archive, implemented and verified credit deduction and dashboard fixes, and prepared the final report structure. Yang Xu also worked with Si Tin Iek on frontend and

backend implementation.

Jiang Xingyu (DC328023)
Focused on the public forum part of the project. Jiang Xingyu asked questions to other teams, answered questions collected from the remaining teams after the presentation, and helped turn the presentation discussion into written responses for the report/forum requirement.

Huang Sofia (DC326312)
Worked on the project PPT. Huang Sofia contributed to slide design, visual storytelling, feature explanation and presentation materials so the project could be communicated clearly during the presentation session.

Fan Zou Chen (DC326351)
Worked on the project PPT together with Huang Sofia. Fan Zou Chen helped organise the slide narrative, presentation flow and deliverable explanation for the team presentation.

Si Tin Iek (DC227126)
Worked with Yang Xu on frontend and backend development. Si Tin Iek supported implementation of the website application and helped connect UI features with service logic.

Ma Iat Tim (DC226328)
Prepared citations and references-related parts of the submission. Ma Iat Tim helped organise APA-style references and ensure external technologies and resources were acknowledged appropriately.

9. Project Criteria of Success

- Functional success: all required service modules can be demonstrated through the website.
- Responsive success: the frontend remains usable on phone, tablet and desktop widths.
- Full-stack success: the source includes both Express/SQLite backend and deployed mock API parity.
- Deployment success: the public URL works without local installation.
- Dashboard success: users can manage profile, credits, search history, cart and orders.
- Peculiar-service success: AI chat, Idea Graph and Hallucination Audit extend the project beyond basic e-commerce.

- Documentation success: proposal, PPT, report, screenshots, references, AI declaration and installation instructions are complete.

10. Project Evaluation

Evaluation Area	Evidence	Self-Evaluation
Functional requirements	Signup/login/reset/dashboard/search/history/cart/order/peculiar services implemented or mirrored by mock API.	Meets requirement
Responsive design	Responsive tests and Ant Design grid-based layouts.	Meets requirement
Deployment	GitHub Pages URL works with mock backend and demo account.	Meets requirement
Testing	3 suites, 18 tests passed.	Strong for course scope
Data model	SQLite tables cover users, products, cart, orders, search history, papers, notes, ideas, chat and audit jobs.	Meets data requirement
Documentation	Report, proposal, PPT, README, screenshots and references.	Meets submission requirement

11. Project Learning - Individual and Team

Member / Team	Learning
Yang Xu	Learned to integrate React frontend, Express backend, SQLite model, mock API and GitHub Pages deployment into one assessable artifact.
Jiang Xingyu	Learned how forum Q&A becomes project evidence, not only presentation activity.
Huang Sofia	Learned how PPT visuals and structure affect technical communication.
Fan Zou Chen	Learned how to support presentation materials and deliverable explanation.
Si Tin Iek	Learned how frontend and backend need shared API contracts and testable flows.
Ma Iat Tim	Learned how citation management supports academic honesty.
Team Learning	Full-stack delivery requires code, requirements mapping, deployment evidence, testing, screenshots, citations, AI-use transparency and role accountability.

12. Project Test Case(s) for Functional Requirements

Test Case	Requirement Covered	Expected / Verified Result
Mock API parity	Static deployment mirrors backend services.	Team, products, paper pagination, auth, cart, orders, ideas, redemption and history return expected results.
Auth flow	Signup, login and forgotten-password recovery.	Register, login, reject wrong password, check account, reset password and login again.
Cart + checkout	Shopping-cart service and order history.	Cart accepts items, checkout creates order and adds credits.
Idea save + unsave	Peculiar Idea Graph service.	Saved ideas appear with related papers; unsaved ideas are removed.
Redemption code	Credit top-up service.	TEAM07-* adds 100 credits; invalid code rejected.
Search history	History service.	Search records are created, deleted and cleared.
Responsive test	Testable responsive frontend.	Home, Dashboard, Paper and Team render at mobile/tablet/desktop widths.

Latest verification: the project test command passed 3 test suites and 18 tests. Production build for GitHub Pages completed successfully with existing ESLint warnings only.

13. Project Demo Walkthrough (Screen Shots)

This walkthrough follows the main user journey and uses screenshots as visual proof of project tasks, insights and learning.

Home and brand entry point. The home page introduces iSuperviz as an AI research supervisor and frames the required full-stack workflow: paper tracking, dashboard, credit store, team information and deployed demo access.

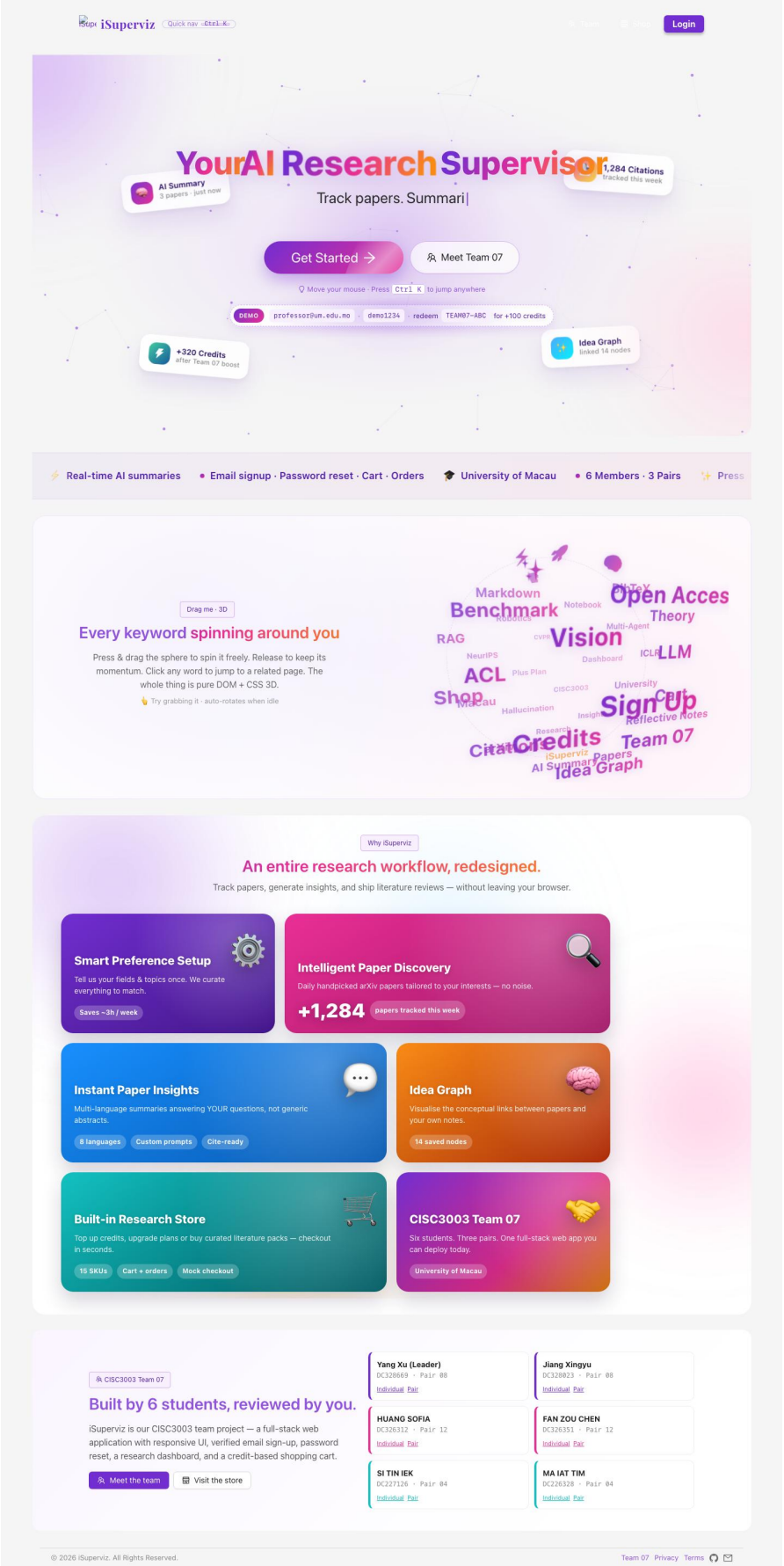

Figure 2. Home and brand entry point.

Team 07 page. The team page lists all six members, pair membership, student IDs, role tags and individual/pair site links used as evidence of team organisation.

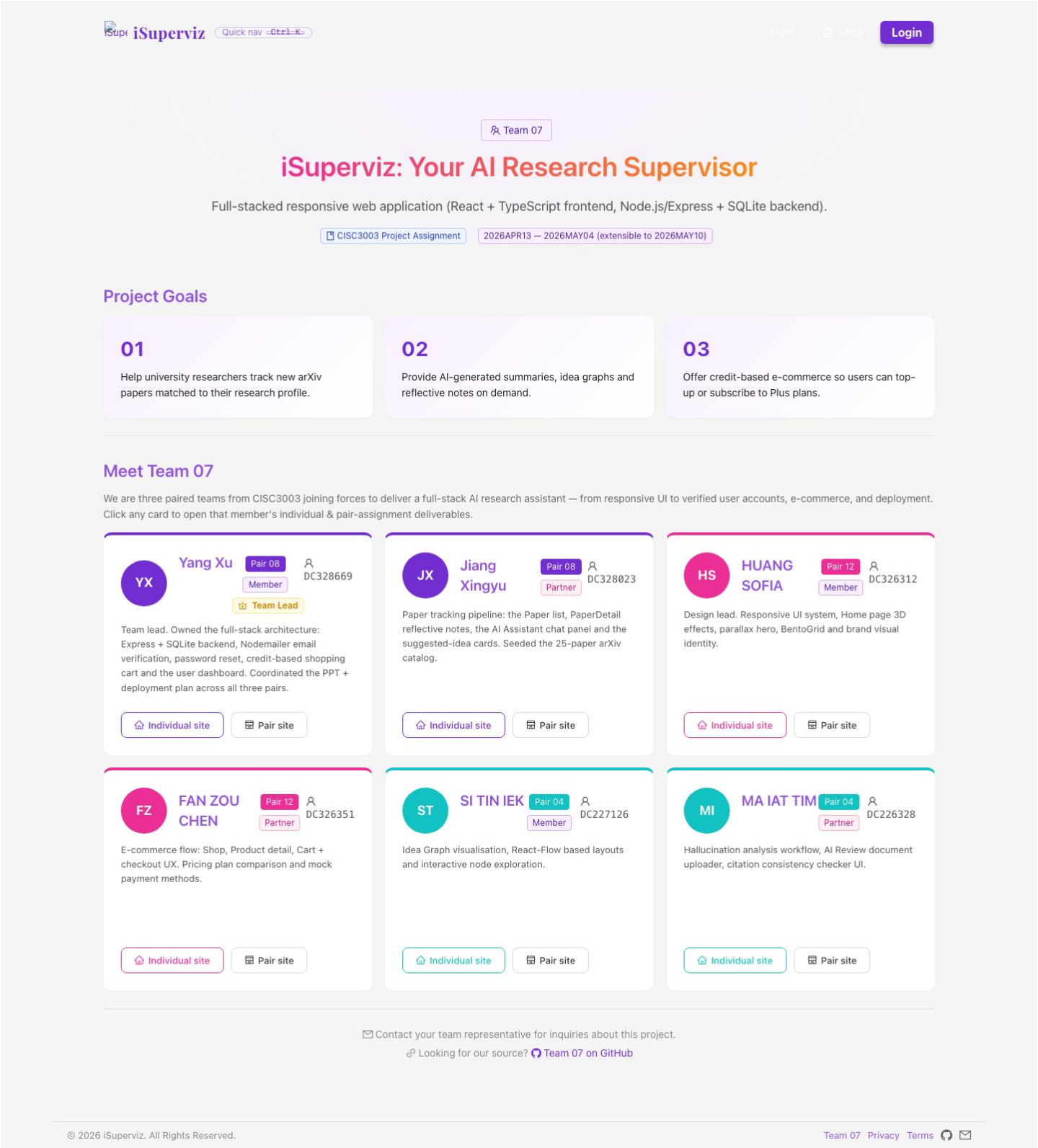

Figure 3. Team 07 page.

Login and sign-up entry. The login page supports the reviewer demo account and the user authentication flow required by the project brief.

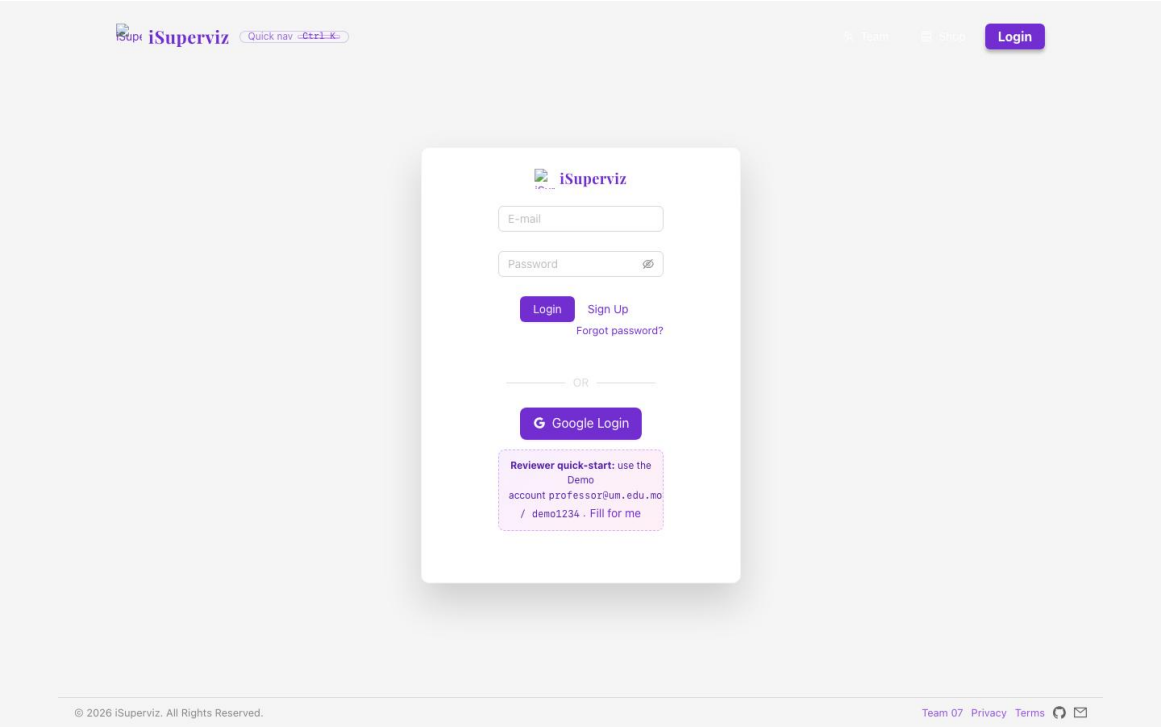

Figure 4. Login and sign-up entry.

Forgotten-password recovery. The password recovery page implements the forgotten-password service module through account checking and reset flow.

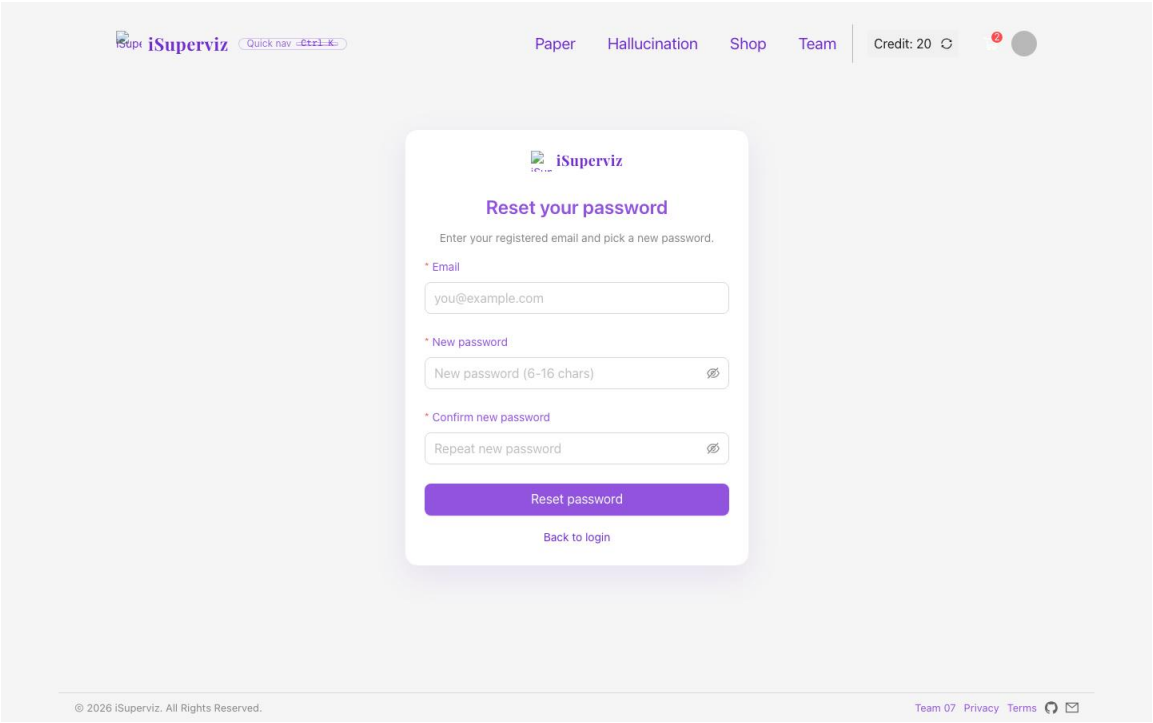

Figure 5. Forgotten-password recovery.

Research preference setting. Users can configure topics, research areas and language preferences so paper recommendations can match the research profile.

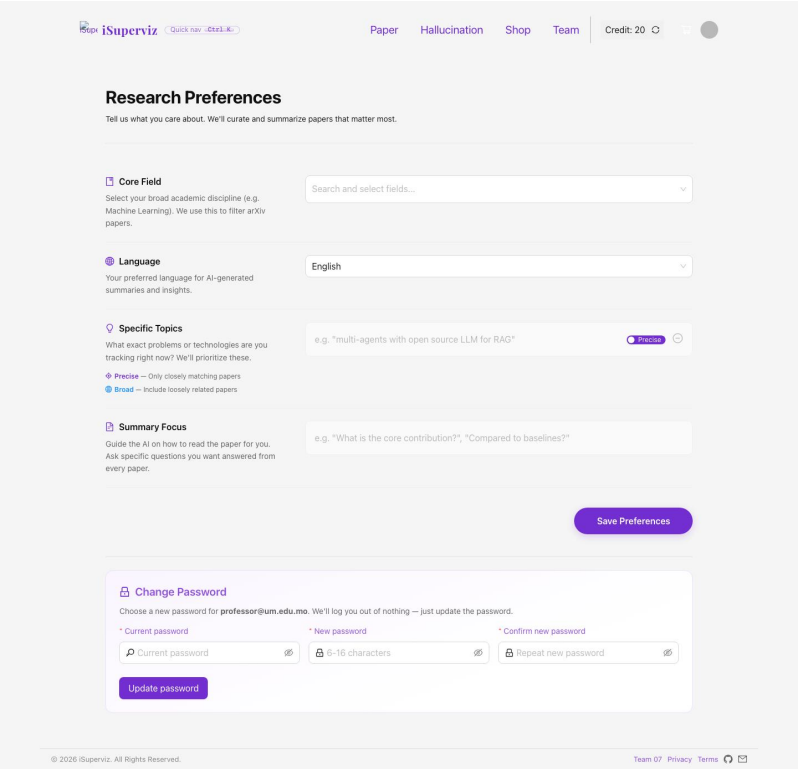

Figure 6. Research preference setting.

Paper tracking list. The paper page provides the search-related service, paper catalogue browsing, favorites, credit-based Explore and PDF download actions.

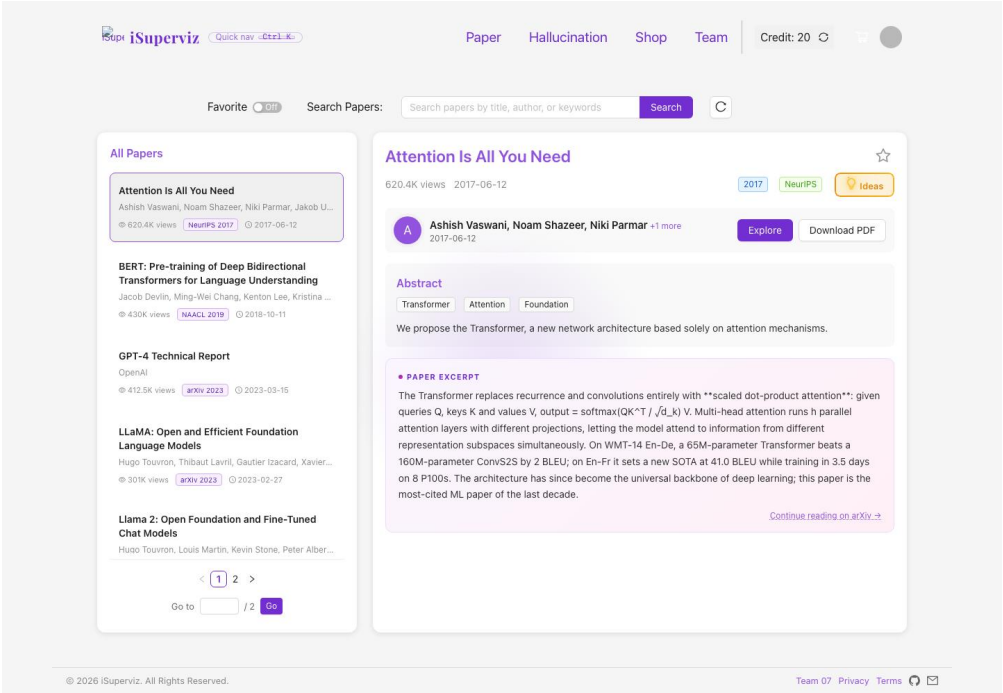

Figure 7. Paper tracking list.

Paper detail and reflective notes. Paper detail gives structured notes, suggested ideas, AI chat and source context for deeper academic reading.

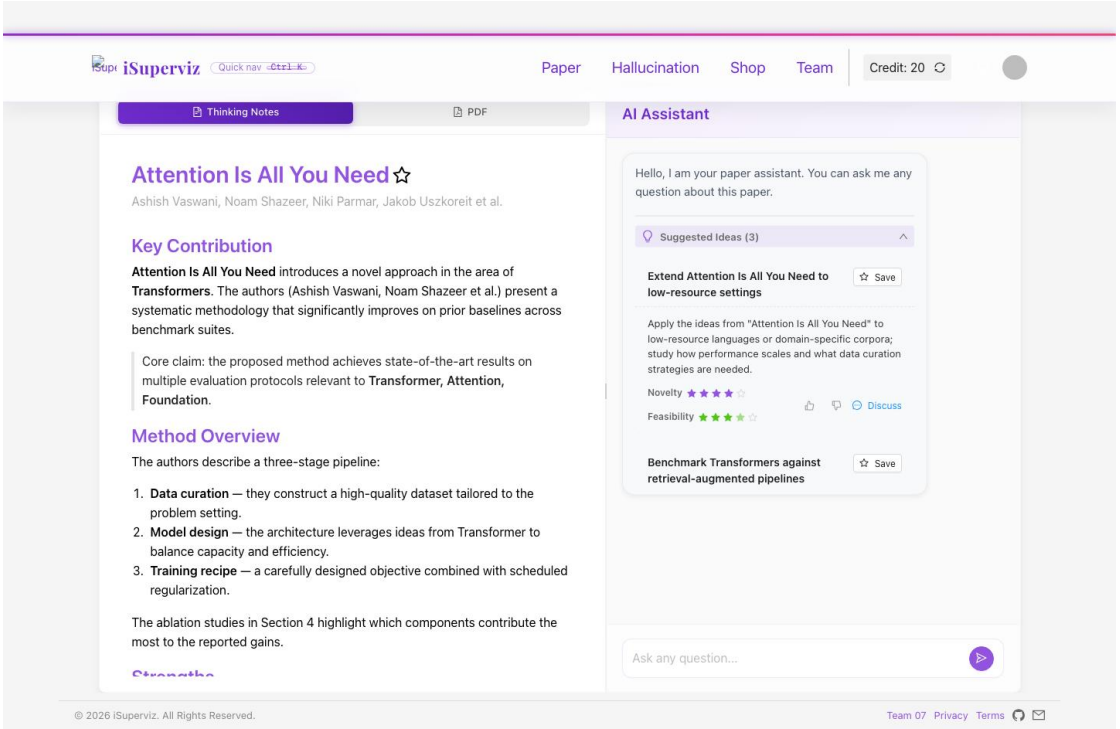

Figure 8. Paper detail and reflective notes.

Hallucination audit peculiar service. The hallucination audit accepts review options and returns a deterministic checking result for AI detection, citation and writing consistency.

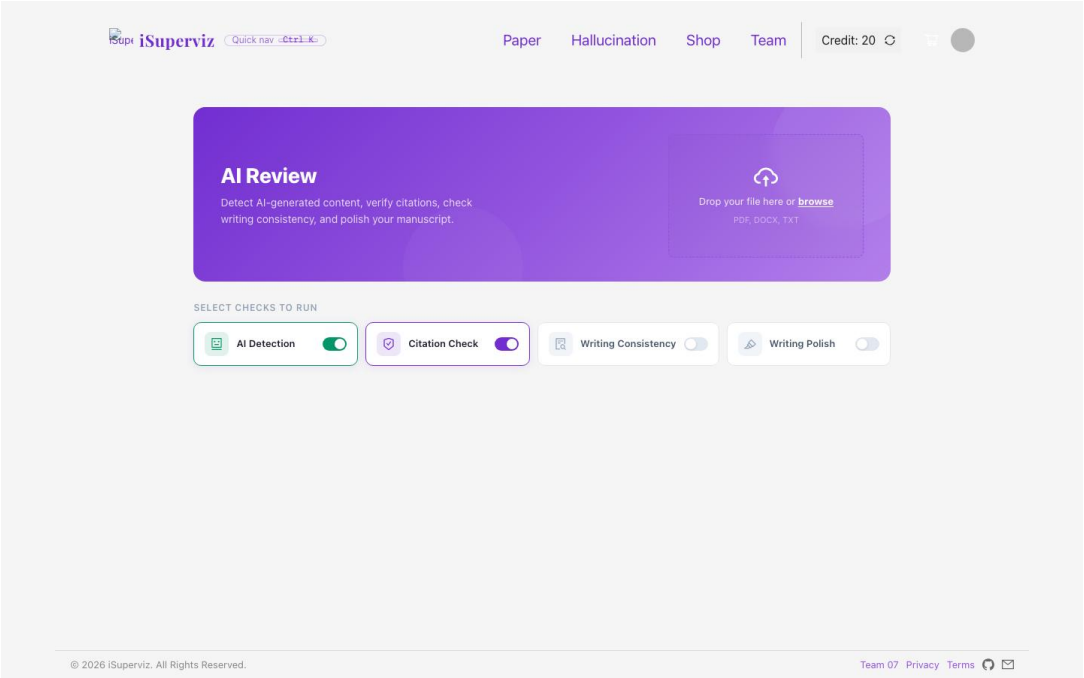

Figure 9. Hallucination audit peculiar service.

Research store. The shop lists credit packs, membership plans, literature review reports and services as the e-commerce part of the project.

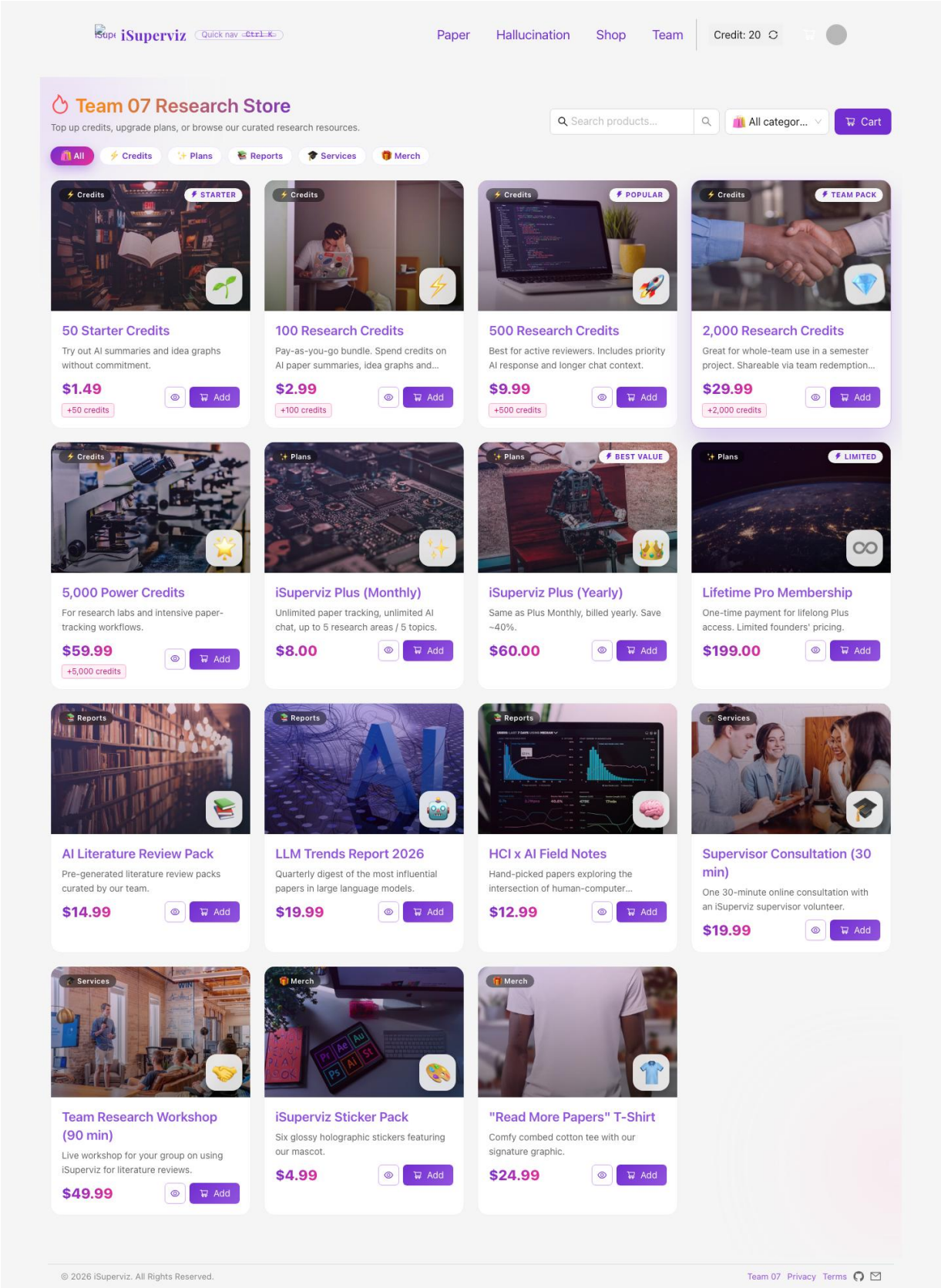

Figure 10. Research store.

Shopping cart and checkout. Cart operations cover add, update, remove, clear and mock checkout, with side effects on credits and membership status.

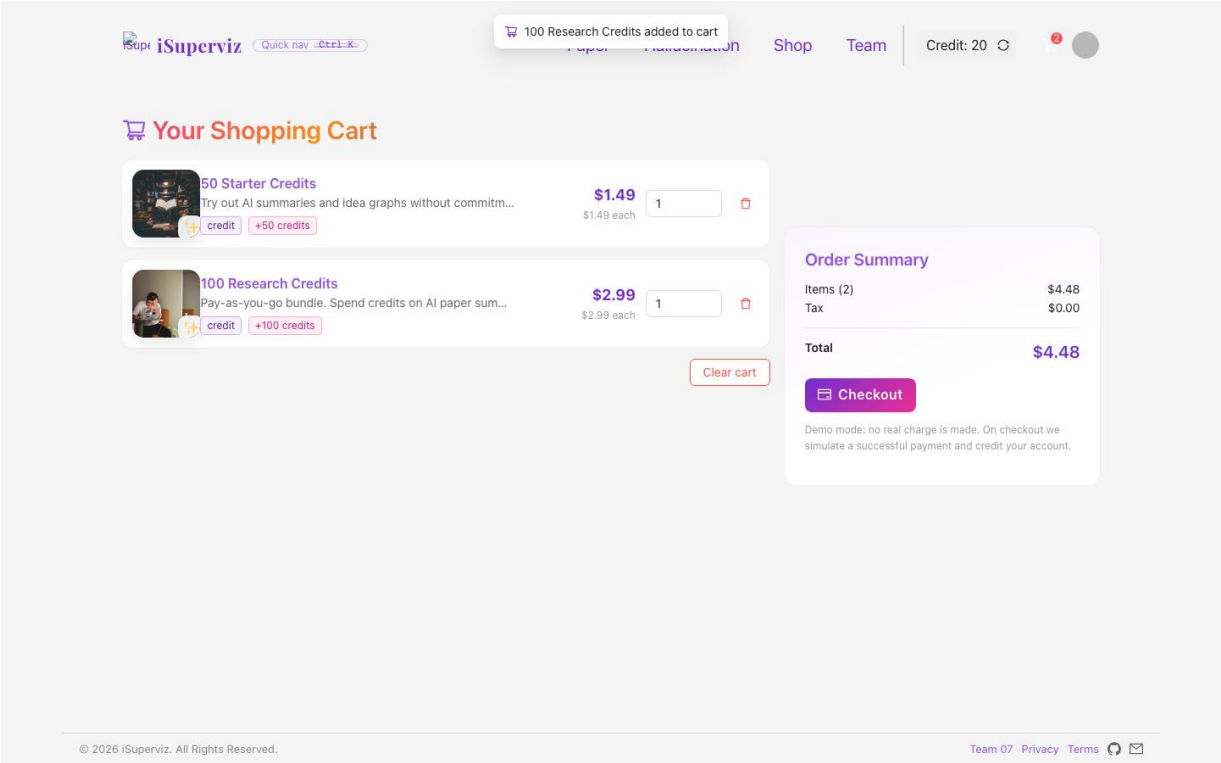

Figure 11. Shopping cart and checkout.

Dashboard, orders and history. The dashboard centralises user profile, credit balance, cart summary, search history and order history for individual activity management.

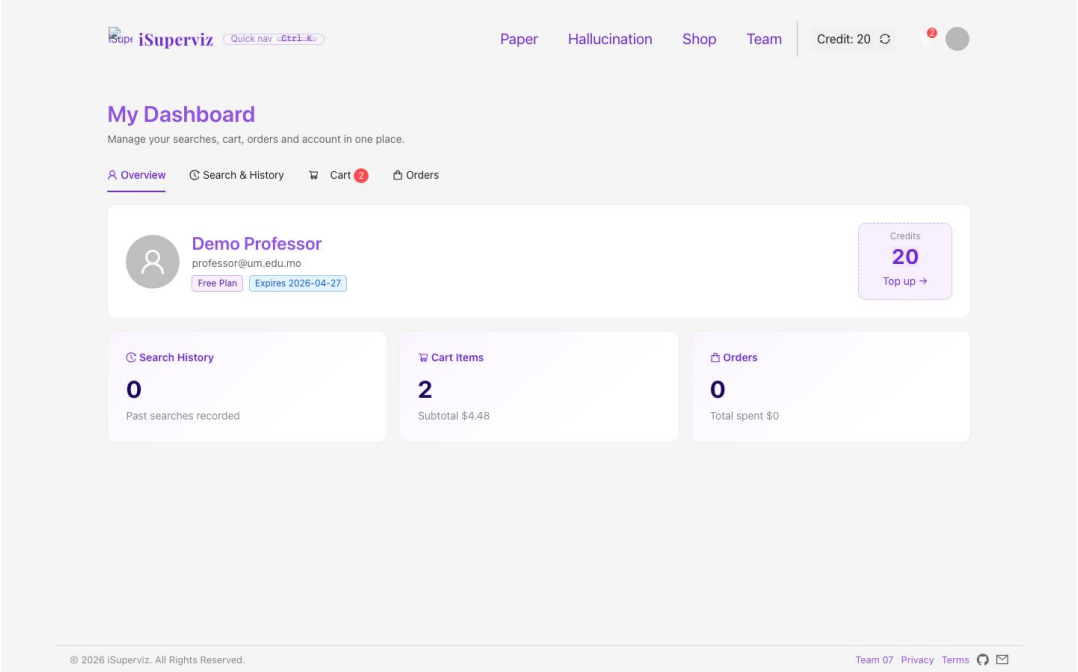

Figure 12. Dashboard, orders and history.

14. Public Forum Q&A Responses from the Remaining Six Teams

The following written responses summarise the questions collected after the presentation session and provide Team 07 responses before the presentation-day forum deadline.

Question from Other Team	Team 07 Written Response
How can it be full-stack if deployed on GitHub Pages?	The source contains a real Express/SQLite backend. GitHub Pages uses a mock API only because Pages cannot run Node.js; the coding archive still demonstrates full-stack design.
How is responsive design verified?	The UI uses Ant Design layouts and includes responsive Jest tests for 360, 768 and 1440 widths plus manual checks.
What data does the system store?	Users, captchas, products, cart items, orders, search history, papers, notes, ideas, related papers, chat messages, feedback and audit jobs.
What is the most important peculiar service?	Idea Graph and Hallucination Audit are the clearest peculiar services beyond standard login/cart modules.
How does the credit system work?	Users top up credits; Paper Explore and PDF download deduct 10 credits; dashboard shows balance and history.
How did the team handle academic honesty?	APA references and AI usage declaration are included; final work was reviewed and edited by the team.

15. Work-in-Progress Details throughout the Project Period for the Collaborative Forum

Project Period	Work-in-Progress Details to Show in Forum
2026 Apr 13-16	Confirmed requirements, selected iSuperviz, mapped service modules.
2026 Apr 17-20	Prepared proposal direction, roster, initial UI and static proof-of-concept.
2026 Apr 21-24	Implemented React pages, backend schema and API contracts.
2026 Apr 25-30	Integrated paper catalogue, PaperDetail, ideas, Idea Graph, Hallucination Audit and store.
2026 May 1-6	Prepared screenshots, PPT, demo walkthrough and forum materials.
2026 May 7-10	Finalised report, refreshed Pages build, checked Team links, credit deduction, dashboard history and archives.

Required forum threads: “Our Project GitHub Link” should contain the GitHub archive URL; “Our Project Deployed URL for Testing” should contain the live URL; WIP details should be posted under Team 07 with the timeline above.

16. Installation Instructions, Deployment Instructions and Data Usage

Local installation and execution:

1. Install Node.js 18/20/22 and npm.
2. Unzip the coding archive or open iSuperviz-full-source-clean.
3. Run npm install in the root.
4. Run npm install --prefix server.
5. Start backend: cd server; PORT=4001 npm start.
6. Start frontend: PORT=8080 REACT_APP_API_TARGET=http://localhost:4001 npm start.
7. Open http://localhost:8080 and login with professor@um.edu.mo / demo1234.

Production build and deployment:

8. Run CI=false REACT_APP_FORCE_MOCK=1 npm run build.
9. Copy build output into CISC3003-TeamProject-Team07-iSuperviz in the GitHub repository.
10. Commit and push to master.
11. Verify GitHub Pages asset-manifest and force-refresh browser if cached.

Data instructions: server/data.sqlite is created automatically. Seed data includes product catalogue, paper catalogue, team roster, demo account and deterministic mock service outputs.

17. Project Incomplete - What Was Not Completed by the Submission Deadline

- The deployed version uses a mock backend because GitHub Pages cannot host Node.js; the real backend is available locally.
- Hallucination Audit uses deterministic demo logic rather than a paid production LLM detector.
- Email verification falls back to console/demo mode unless SMTP credentials are configured.
- The paper catalogue is curated/seeded rather than a live arXiv crawler.
- Digital stories must still be recorded and submitted separately by the required links.
- Further accessibility improvements such as ARIA live regions can be added later.

18. Project Citations and References (APA Style)

Ant Design Team. (n.d.). Ant Design - A design system for enterprise-level products. https://ant.design/

Cornell University. (n.d.). arXiv.org e-Print archive. https://arxiv.org/

Express.js. (n.d.). Express - Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com/

GitHub, Inc. (n.d.). GitHub Pages documentation. https://docs.github.com/en/pages

Hapuarachchi, J. (n.d.). better-sqlite3: The fastest and simplest library for SQLite3 in Node.js. https://github.com/WiseLibs/better-sqlite3

Meta Open Source. (n.d.). React documentation. https://react.dev/

Microsoft. (n.d.). TypeScript documentation. https://www.typescriptlang.org/docs/

Node.js Foundation. (n.d.). Node.js documentation. https://nodejs.org/en/docs/

Nodemailer. (n.d.). Nodemailer - Send emails from Node.js. https://nodemailer.com/

React Flow. (n.d.). React Flow - Build node-based UIs with React. https://reactflow.dev/

SQLite Consortium. (n.d.). SQLite documentation. https://www.sqlite.org/docs.html

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

19. Project Declaration of AI Usage

Team 07 acknowledges the use of AI assistants, including ChatGPT, Claude and Cursor, as assistive tools during the project period. AI was used under these circumstances:

- Code assistance for React/TypeScript components, Express endpoints, mock API parity and debugging.
- Documentation assistance for README, proposal/report wording, PPT narration and checklist mapping.
- Testing and review assistance for missing requirements, route checks, deployment build output and test evidence.
- UI copy assistance for labels, demo instructions and feature descriptions.

All final code, design choices, written content and screenshots were reviewed and edited by the team. AI was not used as a substitute for team judgement, learning or responsibility. No personal or sensitive user data was shared with AI services.

20. Remarks

- The report content is originally written for Team 07 based on the iSuperviz project idea and implementation evidence.
- External technologies and resources are cited in APA style.
- Demo account: professor@um.edu.mo / demo1234; redemption code: TEAM07-ABC.

Final Requirement Coverage Checklist

Required Item	Coverage Status
Title page with team/project/members	Covered
300-word abstract	Covered
TOC	Covered
Context: full-stack RWD deployed URL	Covered
Goals	Covered
Services and data requirements	Covered
Tasks and tools	Covered
Challenges	Covered
Deliverables	Covered
Division of work	Covered
Individual contributions	Covered
Criteria of success	Covered
Evaluation	Covered
Individual + team learning	Covered
Functional test cases	Covered
Demo walkthrough screenshots	Covered
APA citations and references	Covered
AI usage declaration	Covered
Installation/deployment/data instructions	Covered
Project incomplete	Covered
Public forum Q&A responses	Covered
WIP forum details	Covered